

Records API

Переосмысление GraphQL
для low-code платформы

С чего все начиналось

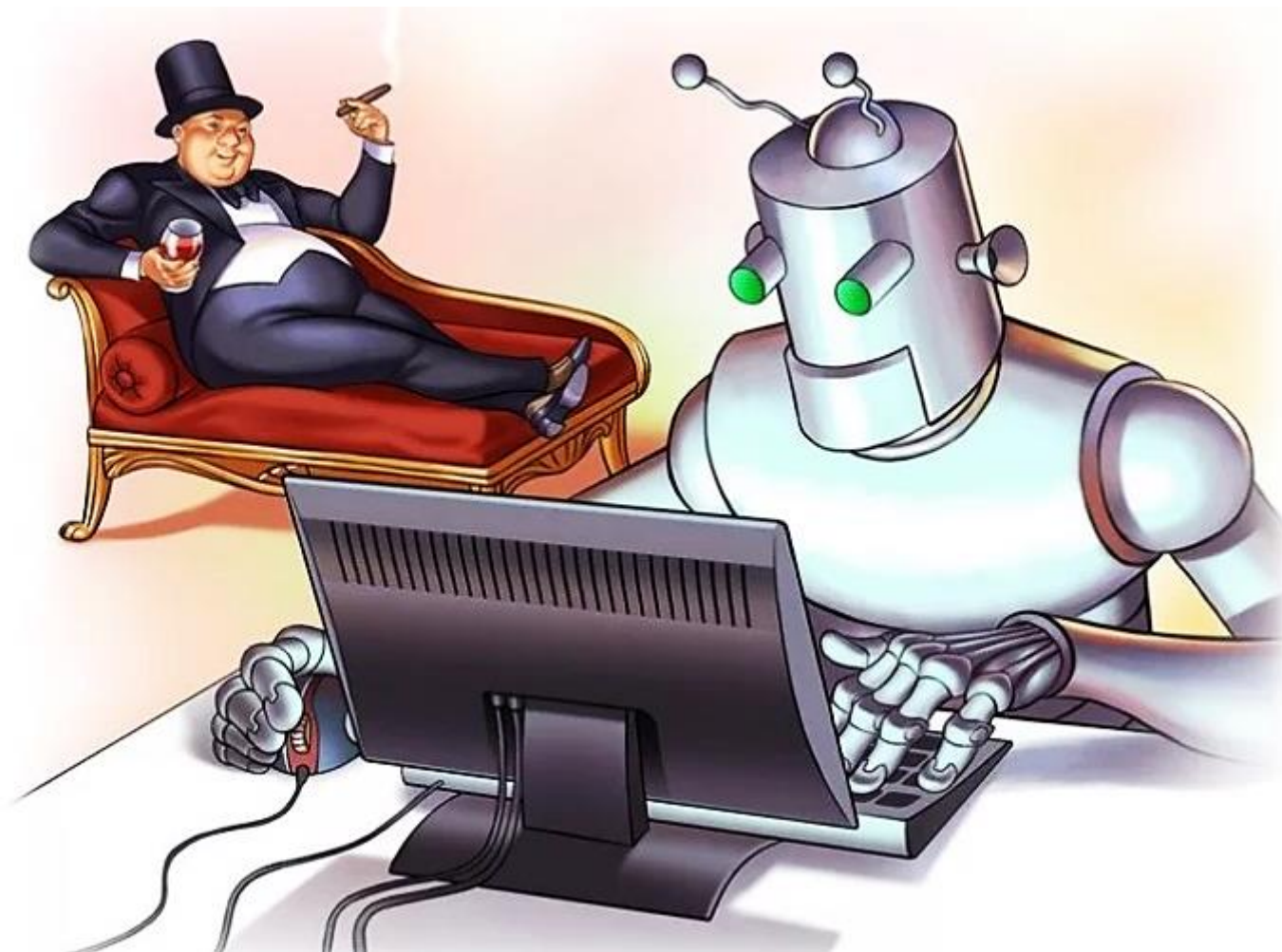
A long time ago in a galaxy far,
far away....

Overfetching



И где тут email
пользователя?

Low-code платформа



Создание нового типа данных

Создать Описание типа данных 

Заккрыть X

Основные

Атрибуты

Аспекты

Роли

Статусы

Стадии

Варианты создания

Id *

attorney

Имя

Доверенность



Создание нового типа данных

< Основные **Атрибуты** Аспекты Роли Статусы Стадии Варианты созд >

− Атрибуты

	Список атрибутов				
≡	Id * name Имя Имя 	Тип Text × ▼	☐ Множественный ☐ Обязательный	 	
≡	Id * number Имя Номер 	Тип Number × ▼	☐ Множественный ☐ Обязательный	 	

+ Добавить еще

Создание нового типа данных

Настройки меню

Журнал шаблонов меню →

Заккрыть X

Настройки выбранной кон...

Меню "Создать"

Пользовательское меню

Глобальные настройки

ID меню: cfe9a91b-bdc3-4c3b-94de-76220b402562

Элементы меню

+ Добавить ▾

Развернуть все

> Задачи

▼ Разделы + Добавить ^

>  c

>  г

>  s

> Оргструкт

> Справочники

Раздел

Разделяющий заголовок

Журнал

Произвольный пункт меню

Ссылку на создание кейса

Запустить бизнес-процесс

Создание нового типа данных

Выбрать: Журналы

Создать ▾

Фильтр ▾

🔍

Доверенность

☒ ▾	Идентификатор	Тип ECOS	Название
☒	type\$attorney	Доверенность	Доверенность

1-1 из 1

< >

Отмена

ОК

Создание нового типа данных

The screenshot displays the Citeck ecos application interface. The top navigation bar includes a menu icon, a '+ Создать' (Create) button, a search bar, a settings gear, a language selector (Рус), a user profile icon, and the role 'Administrator'. The left sidebar shows the 'Доверенность' (Trust) section selected, with other options like 'Задачи' (Tasks), 'Разделы' (Sections), 'Оргструктура' (Organizational Structure), and 'Справочники' (Reference Lists). The main content area shows the 'Доверенность' section with a '+', a settings gear, a search bar, and a table with columns 'Имя' (Name) and 'Номер' (Number). A blue arrow points to the '+' icon, indicating the action to create a new data type.

Создать

Найти файл, пользователя или раз

Рус

Administrator

Тип "Доверенность" X Журнал "Доверенность" X X

Доверенность ⚙

← Показать меню

+ ⚙

Найти

Действия над 0

Экспорт

Имя

Номер

Нет элементов в списке

Создание нового типа данных

Создать Доверенность 

Заккрыть X

Имя

Доверяю свою одежду и мотоцикл

Номер

1

Отменить

Сохранить

Создание нового типа данных

Доверяю свою одежду и мотоцикл

Основное

Предпросмотр

История

Документы

^ Мои задачи 0



^ Свойства



Имя Доверяю свою одежду и мотоцикл

Номер 1

^ Комментарии



Нет комментариев

Добавить комментарий

Статус Нет статуса

^ Все задачи 0



^ Действия

Редактировать Свойства

Создать поручение

^ Связи докумен Refresh +

Создание нового типа данных

Доверенность 

← Показать меню

+



 Найти

Действия над 1 

Экспорт 





☐ 	Имя 	Номер 
☐	Доверяю свою одежду и мотоцикл	1

1-1 из 1



10 

Создание нового типа данных

```
▼  t_attorney
  ▼  columns 15
     __creator bigint
     __created timestamp with time zone
     __modifier bigint
     __upd_version bigint
     __type bigint
     __deleted boolean
     __name varchar
     __modified timestamp with time zone
     __ext_id varchar
      id bigint = nextval('ecos_data...')
     __ref_id bigint
     __status varchar
     name varchar
     number double precision
     _isDraft boolean
```

GraphQL



GraphQL не подошел



Oh no

Минусы GraphQL для low-code платформы

1. Обязательность схемы;
2. Работа универсальных клиентов затруднительна;
3. Бэкенд тоже может быть клиентом;
4. Формат GraphQL не так популярен как хотелось бы.

Плюсы GraphQL

1. Клиент точно знает тип результатов, которые он получит из запроса;
2. Нет underfetching/overfetching проблем;
3. Единый Endpoint для работы;

Records API

- **Микросервис** – это некоторое приложение с уникальным именем в пределах платформы;
- **Запись** (Record) – это любой ресурс в системе;
- **Источник данных** – это DAO для **записей**, который реализует все или некоторые CRUD операции. Каждый **источник данных** имеет уникальный идентификатор в пределах **микросервиса**;
- Каждая **запись** имеет глобальный идентификатор, который содержит имя **микросервиса**, локальный идентификатор **источника данных** в пределах **микросервиса** и локальный идентификатор записи в пределах **источника данных**.

emodel/person@admin

Records API Query

Где ищем?

sourceId: `String` (emodel/person, eproc/tasks, ...)

Как ищем?

language: `String` (predicate, sql, ...)

Что ищем?

query: `Any`

По каким атрибутам группируем?

groupBy: `String[]`

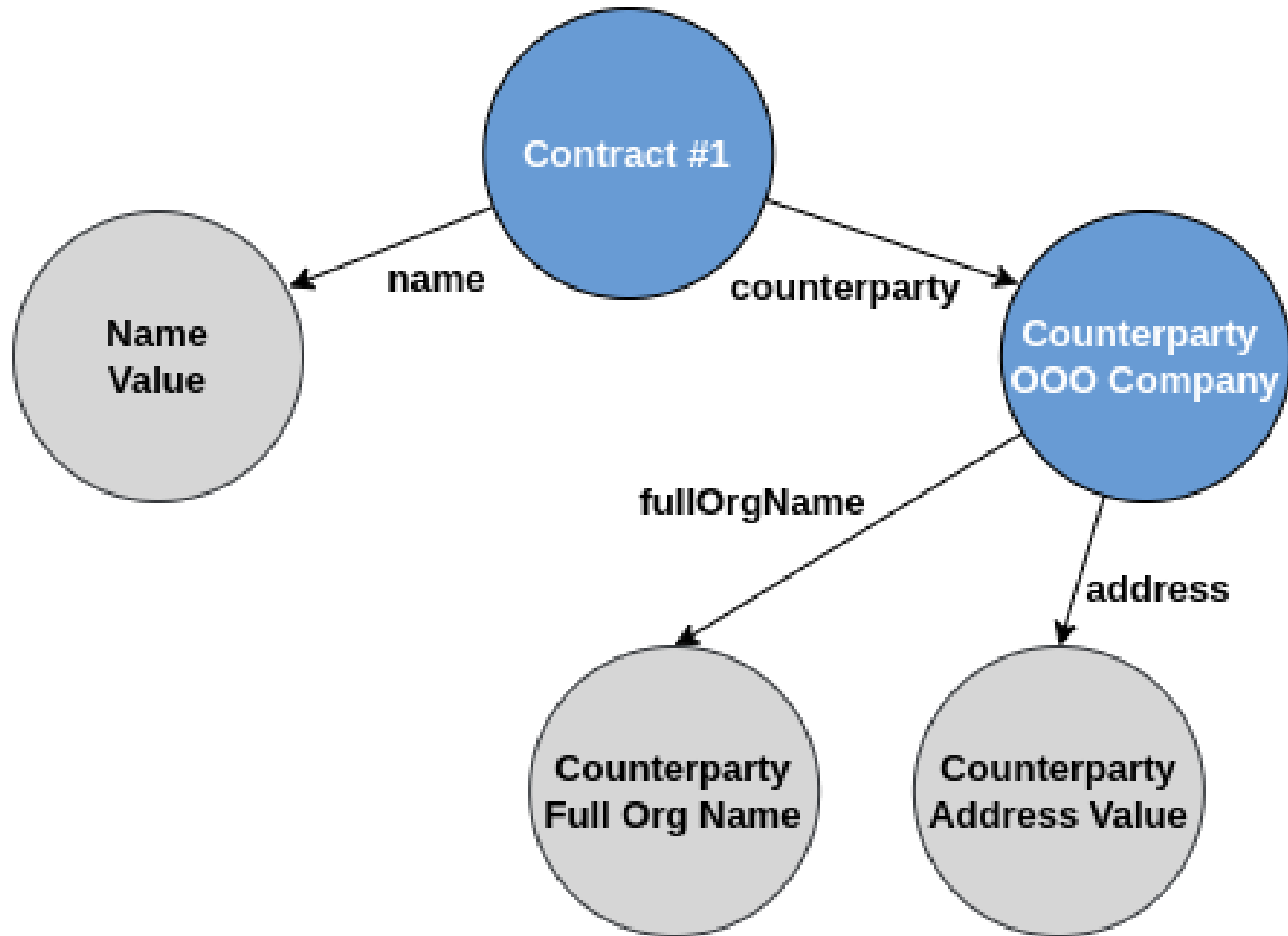
Как сортируем?

sortBy: `SortBy[]` { **attribute:** `String`, **ascending:** `Bool` }

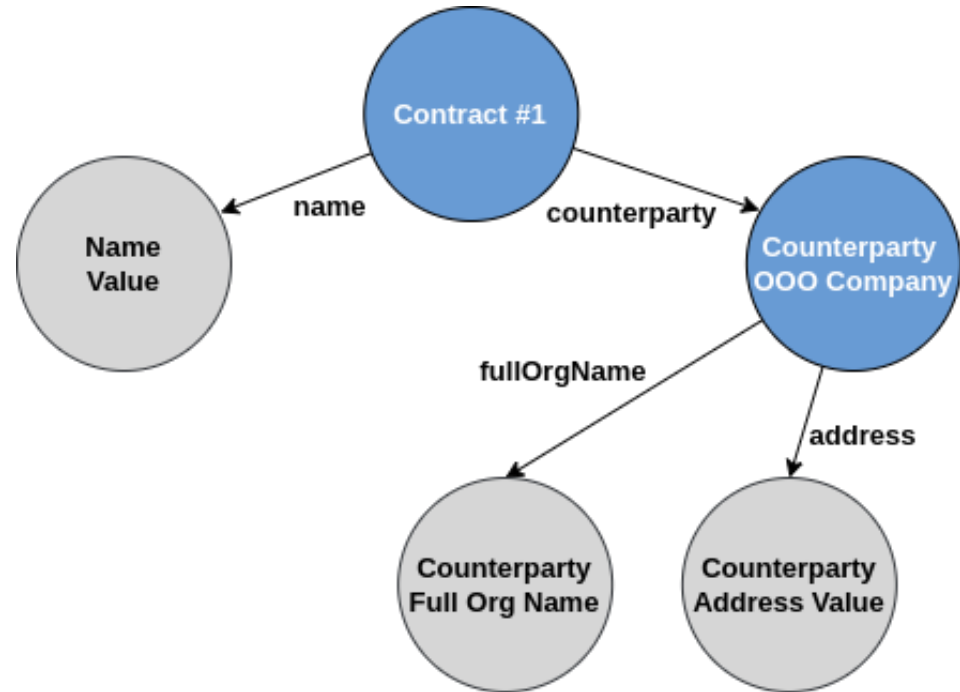
Сколько пропустить и сколько взять?

page: `Page` { **skipCount:** `Int`, **maxItems:** `Int` }

Атрибуты



Атрибуты



Contract #1

```
{  
  "id": "e0c554"  
  "name": "Contract with OOO Company",  
  "counterparty": "emodel/counterparty@177f68"  
}
```

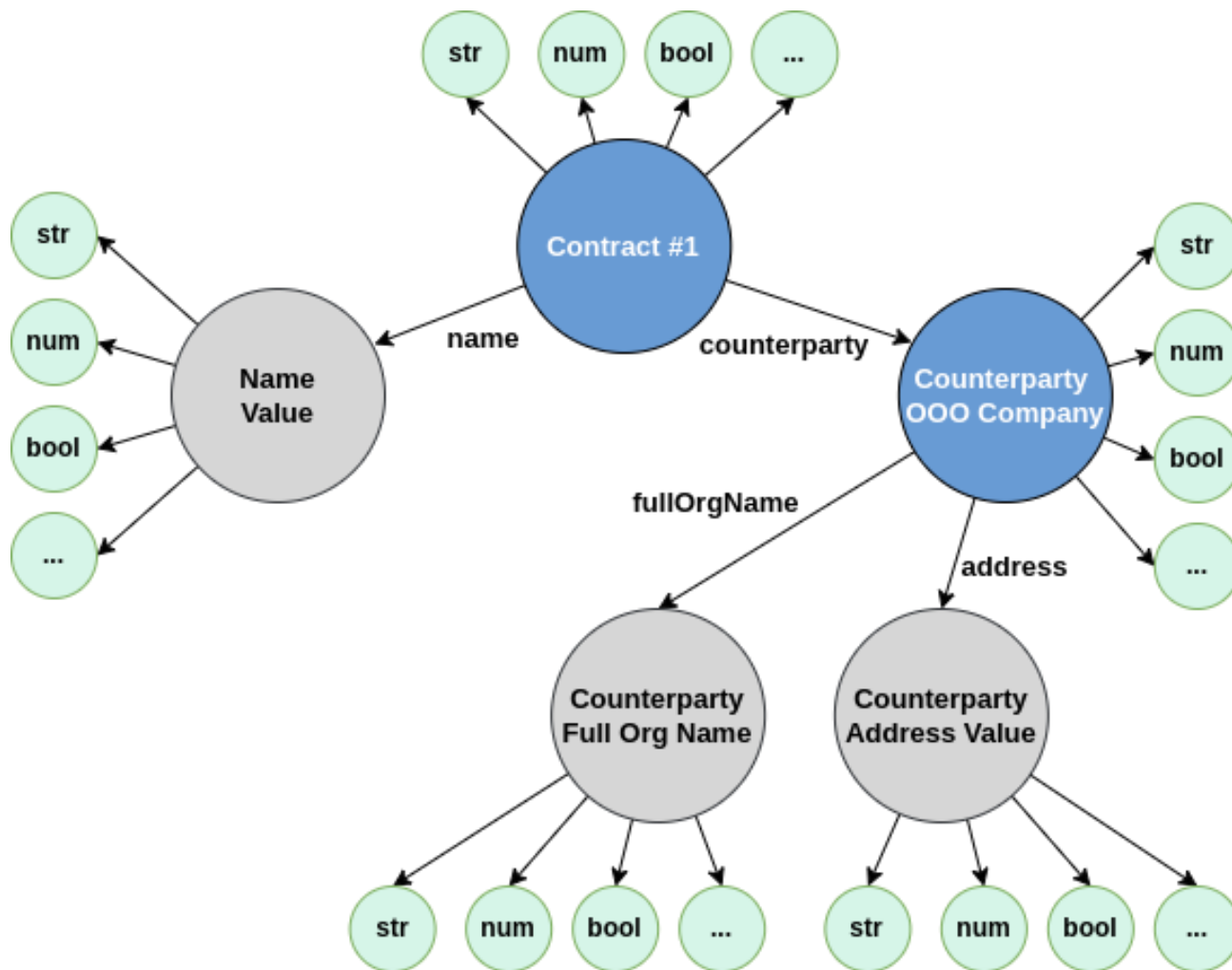
Counterparty "OOO Company"

```
{  
  "id": "177f68",  
  "fullOrgName": "OOO Unlimited Company",  
  "address": "Moscow, Pushkina st."  
}
```

association0.association1.someField

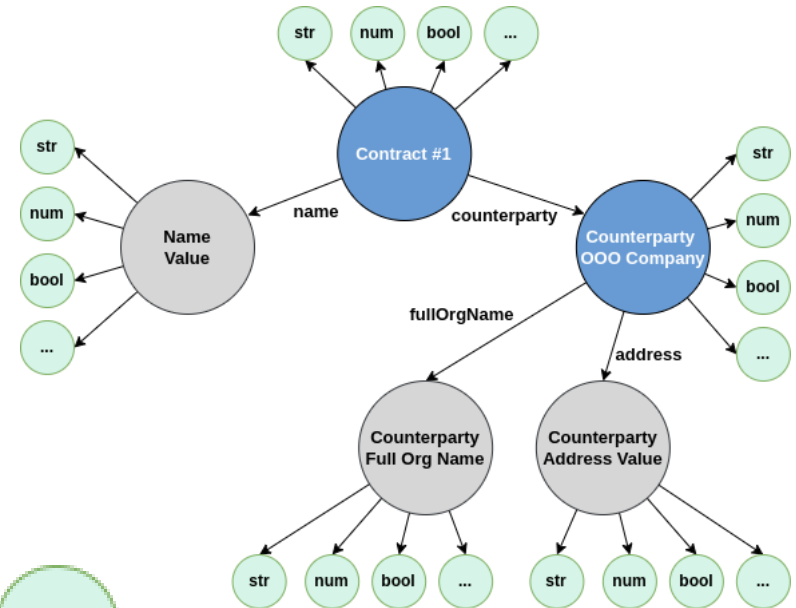
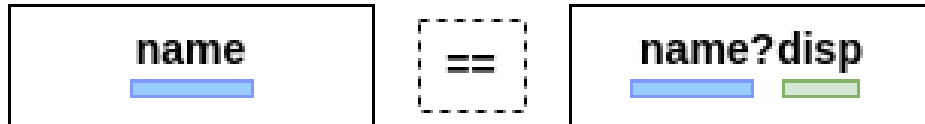
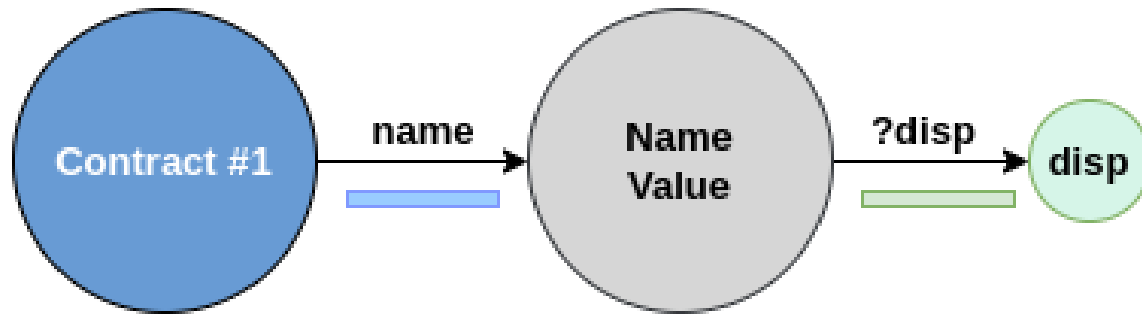
counterparty.fullOrgName

Скаляры

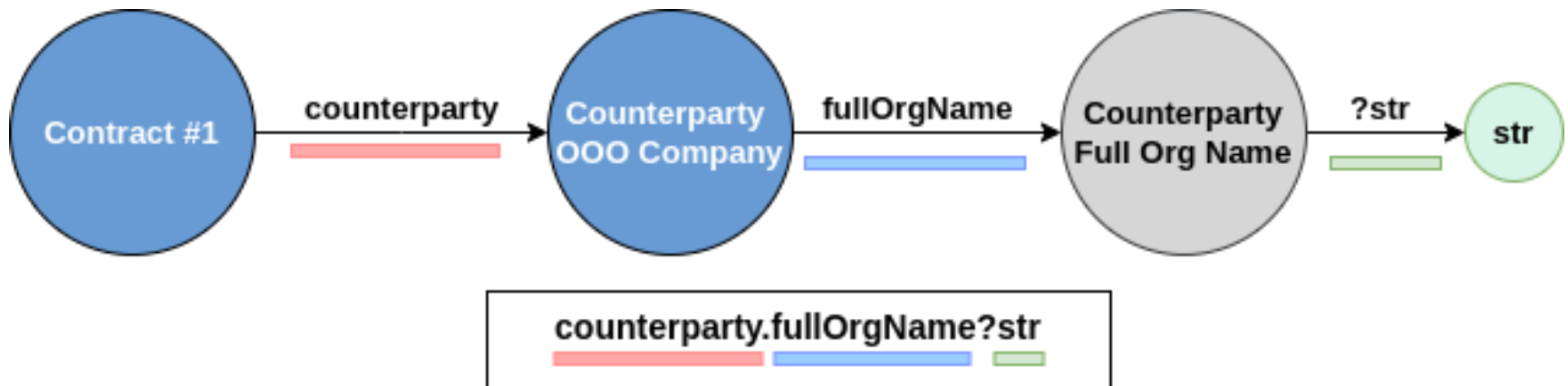
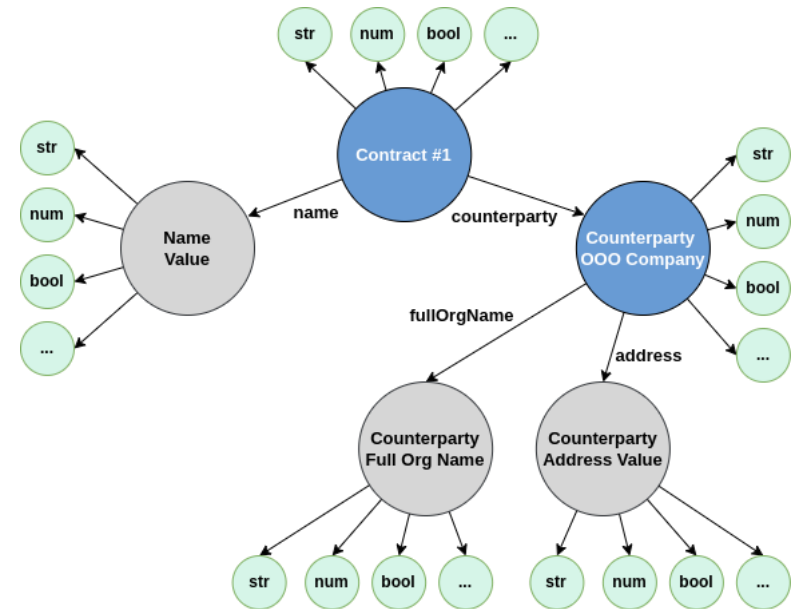


association0.association1.someField?scalar

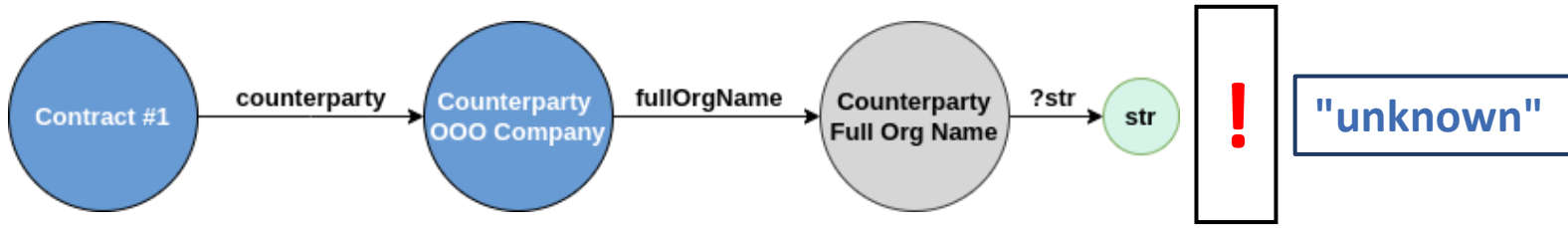
Загрузка имени договора



Загрузка полного имени контрагента



Пост-процессоры атрибутов



counterparty.fullOrgName?str!"unknown"

==

counterparty.fullOrgName?str |or("unknown")

attribute |processor0(arg0,arg1,argN)|processor1(arg0,arg1,argN)

HTTP API. Поиск записей

POST /gateway/records/query

```
{  
  "query": {  
    "sourceId": "emodel/person",  
    "language": "predicate"  
    "query": { "t": "eq", "a": "email", "v": "admin@citeck.ru" },  
    "page": { "maxItems": 1 }  
  },  
  "attributes": {  
    "userId": "?localId"  
  }  
}
```

HTTP API. Поиск записей

Ответ

```
{
  "records": [
    {
      "id": "emodel/person@admin",
      "attributes": {
        "userId": "admin"
      }
    }
  ],
  "totalCount": 1
}
```

HTTP API. Получение атрибутов

POST /gateway/records/query

```
{  
  "records": [  
    "emodel/person@admin",  
    "emodel/person@pushkin"  
  ],  
  "attributes": {  
    "email": "email!undefined",  
  }  
}
```

HTTP API. Получение атрибутов

Ответ

```
{
  "records": [
    {
      "id": "emodel/person@admin",
      "attributes": {
        "email": "admin@citeck.ru"
      }
    },
    {
      "id": "emodel/person@pushkin",
      "attributes": {
        "email": "undefined"
      }
    }
  ]
}
```

HTTP API. Изменение записей

POST /gateway/records/mutate

```
{
  "records": [
    {
      "id": "emodel/person@admin",
      "attributes": {
        "email": "new-email@citeck.ru",
        "address": "Moscow, Pushkina st."
      }
    }
  ]
}
```

HTTP API. Изменение записей

Ответ

```
{  
  "records": [  
    {  
      "id": "emodel/person@admin",  
      "attributes": {}  
    }  
  ]  
}
```

HTTP API. Создание записей

POST /gateway/records/mutate

```
{
  "records": [
    {
      "id": "emodel/person@",
      "attributes": {
        "id": "pk.simonov"
        "email": "pavel.simonov@citeck.ru",
        "address": "Tomsk, Pushkina st."
      }
    }
  ]
}
```

HTTP API. Создание записей

Ответ

```
{  
  "records": [  
    {  
      "id": "emodel/person@pk.simonov",  
      "attributes": {}  
    }  
  ]  
}
```

HTTP API. Удаление записей

POST /gateway/records/delete

```
{  
  "records": [  
    "emodel/person@admin",  
    "emodel/person@pushkin"  
  ]  
}
```

Ответ

```
{  
  "statuses": ["OK", "OK"]  
}
```

Можно попробовать?

Конечно да!
Вот демо проект:



<https://github.com/Citeck/ecos-records-demo>

Спасибо за внимание!

Документация
по Records API



https://citeck-ecos.readthedocs.io/ru/latest/general/ECOS_Records.html